

Como implementar infraestrutura como código

Adequar versão do JDK

Utilizar o Java 21. É uma versão LTS (Long Term Support), com correções de segurança de longo prazo, suportada pelo Spring Boot 3.2 em diante e pelo Spring Boot 4.

pom.xml

```
<properties>
    <java.version>21</java.version>
    ...
</properties>
```

Pontos de atenção:

- Ao subir a versão do JDK, ajustar também a imagem base do Dockerfile e alinhar com o time de DevOps, pois a imagem usada pela pipeline precisa acompanhar.
- Evitar versões que não sejam LTS (como o Java 26): elas deixam de receber correções assim que a versão seguinte é publicada, a cada seis meses.
- Projetos que ainda estejam no Java 17 continuam funcionando — o Spring Boot 4 mantém o Java 17 como mínimo. A atualização para o 21 é recomendada, não obrigatória, e pode ser feita separadamente da migração do Spring para não concentrar risco.

Documentação da API

Swagger

```
Criar um novo pacote para o SpringDoc.

Criar a classe SpringDocConfigurations.java

Adicionar as tags necessárias do SpringDoc no Controller
```

Postman

```
Criar uma collection.

Adicionar as rotas necessárias.
```

Bug Tracker Sentry

Adicionar o Sentry no pom.xml:

```
<dependency>
    <groupId>io.sentry</groupId>
    <artifactId>sentry-spring-boot-starter-jakarta</artifactId>
    <version>x</version>
</dependency>
<dependency>
    <groupId>io.sentry</groupId>
    <artifactId>sentry-logback</artifactId>
    <version>x</version>
</dependency>
```

Inserir no application.properties (adequar o nome das variáveis de ambiente, alinhar com time de DevOps):

```
# =====
#  Sentry Config
# =====
sentry.dsn=${SENTRY_DSN_SISMCOM}
sentry.environment=${SENTRY_ENVIRONMENT_SISMCOM}

# Logging - https://docs.sentry.io/platforms/java/guides/spring-boot/logging-frameworks/
sentry.logging.enabled=true
sentry.logging.minimum-event-level=${SENTRY_LOG_LEVEL_SISMCOM}
sentry.logging.minimum-breadcrumb-level=debug

# Sampling - https://docs.sentry.io/platforms/java/guides/spring-boot/configuration/sampling/
sentry.sample-rate=0.5
```

Adicionar API no Supervisório

Adicionar no pom.xml:

```
<dependency>
    <groupId>de.codecentric</groupId>
    <artifactId>spring-boot-admin-starter-client</artifactId>
    <version>x</version>
</dependency>
```

Inserir no application.properties (adequar o nome das variáveis de ambiente, alinhar com time de DevOps):

```
management.endpoint.health.show-details=always
management.endpoints.web.exposure.include=details,health,metrics,beans,mappings,cache,threaddump, heapdump,loggers,configprops,scheduledtasks
info.app.name=@project.name@
info.app.version=@project.version@

spring.boot.admin.client.url=${SUPERVISOR_PATH}
spring.boot.admin.client.instance.service-base-url= ${SISMCOM_PATH}
```

Caso esteja usando Spring Security é necessário liberar as seguintes rotas como exceção:

```
antMatcher("/actuator"),
antMatcher("/actuator/**")
```

Exemplo para Spring Boot 3.x. O `AntPathRequestMatcher`, usado através do `antMatcher()`, foi removido no Spring Security 7, que acompanha o Spring Boot 4. Este trecho será atualizado após a primeira migração real para o Boot 4.

Containerização da API

Para a criação dos arquivos de Dockerfile e docker-compose.yml, solicitar ao time de DevOps.

Variáveis de ambiente para campos chave

Boas práticas de nomenclatura de variável de ambiente

Utilizar o caracter underline "_" para separar uma palavra da outra.

Exemplo:

```
EBS_API_KEY=
SENTRY_LOG_LEVEL_PROCESS_CPR_API=
ORACLE_DB_CONNECT_STRING=
```

Boas práticas para referenciar às variáveis de ambiente

Como boa prática de desenvolvimento, solicitamos o apoio de todos os desenvolvedores para referenciarem as variáveis de ambiente **apenas através do application.properties**.

Isso facilitará futuras manutenções no código-fonte e inclusive facilita o entendimento da aplicação, já fornecendo uma visão de quais são os valores que estão externos à aplicação.

Para isso, referenciar as variáveis através do valor contido no application.properties (exemplo: \${http.auth-token})e no application.properties atribuir um alias (exemplo: \${API_KEY}); que deverá ser o valor que estará no .env.

Exemplo de código na classe:

```
public class APISecurityConfig {

    @Value("${http.auth-token-header-name}")
    private String principalRequestHeader;

    @Value("${http.auth-token}")
    private String principalRequestValue;

}
```

Exemplo de código no application.properties:

```
http.auth-token-header-name=x-api-key
http.auth-token=${API_KEY}
```

Boas práticas para variáveis de ambiente no application.properties

Contamos com o apoio de todos para não incluir nenhuma senha, mesmo que seja mocada, na frente da variável de ambiente no application.properties.

Exemplo do que não fazer:

```
http.auth-token=${API_KEY:teste123456}
```

Manter da seguinte forma:

```
http.auth-token=${API_KEY}
```

Diretório de logs

Criar uma pasta de log em sua aplicação na raiz e adicionar no application.properties o diretório onde ficarão salvos os logs (adequar nome do log conforme o nome da aplicação):

```
# habilitar o modo lazy nas consultas
spring.jpa.open-in-view=false

logging.file.name = ./log/sismcom.log
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE
```

Adicionar nas suas classes a anotação "@Slf4j" e incluir conforme necessidade o comando:

```
log.info(" ##### Realizando busca com os parametros...)
```

Arquivo .gitignore

Criar arquivo .gitignore no projeto para não subir coisas desnecessárias para o projeto no GitLab.

Incrementar no seu projeto com os demais diretórios e arquivos que julgar necessário.

```
/target/*
!.mvn/wrapper/maven-wrapper.jar
!**/src/main/**/target/
!**/src/test/**/target/
!**/target/**/build/
!Library/nao_ignorar Library/
.env

*.log
*.tar.gz
*.gz
### STS ###
.appt_generated
.classpath
.factorypath
.project
.settings
.springBeans
.sts4-cache

### IntelliJ IDEA ###
.idea
*.iws
*.iml
*.ipr

### NetBeans ###
/nbproject/private/
/nbbuild/
/dist/
/nbdist/
/.nb-gradle/
build/
!**/src/main/**/build/
```

```
!**)src/test**)build/

### VS Code ###
.vscode/
```

Arquivo .env.example somente com as variáveis

Como o arquivo de variável de ambiente .env não é versionado, criar uma cópia dele com o nome .env.example e remover **todas as senhas desse arquivo**.

Adicionar comentários informando o porquê da variável, por exemplo:

```
# Setting for EBS Oracle database:
ORACLE_DB_CONNECT_STRING=
ORACLE_DB_USER=
ORACLE_DB_PWD=

# Supervisor application path:
SUPERVISOR_PATH=

# Path to communicate with Supervisor application:
SYSTEM_PATH=

# Setting for Sentry environment:
SENTRY_DSN_CECAFE_API=
SENTRY_ENVIRONMENT_CECAFE_API=
SENTRY_LOG_LEVEL_CECAFE_API=
```

Atualizar arquivo README.md

Atualizar o arquivo README.md do projeto com o desenho da arquitetura da API.

Projeto de referência: [modok-api](#)

Criar arquivo IM090.md

Criar o arquivo IM090.md com as instruções a serem utilizadas para o repasse.

Projeto de referência: [modok-api](<https://gitlab.cooxupe.com.br/desenvolvimento/integracao/spring-boot/tecnologia-da-informacao/controle-de-usuarios-no-active-directory/modok-api/-/blob/development/IM090.md>)

Adequação do nome do artefato

Adequar no pom.xml o artifactId, version, name e description, conforme exemplo:

```
...
    <groupId>br.com.cooxupe</groupId>
    <artifactId>modok-api</artifactId>
    <version>1</version>
    <name>modok-api</name>
    <description>Api para integração com o sistema AD</description>
...

```

Adicionar a tag "FinalName" para gerar o artefato com o nome correto:

```
...
    <build>
        <finalName>${project.artifactId}</finalName>
        ...
    </build>
...

```

Catálogo de APIs no EBS

Cadastrar a nova aplicação no Catálogo de API no banco de dados do EBS.

EBS de desenvolvimento -> cadastrar API de desenvolvimento EBS de homologação -> cadastrar API de homologação EBS de produção -> cadastrar API de produção

Colocar o script acima para ser executado no patch ou no datafix.